

Software Design

Phoenix Automation Framework

P2-Sal Interface Error Handling

Document Number: xxx-xxxx-xxx

Revision 1.0

April 14, 2006

Approvals:

Prepared by: Kevin Guerra Senior Software Developer	Prepared by:
TBD Name TBD Title	TBD Name TBD Title
TBD Name TBD Title	TBD Name TBD Title

REVISION HISTORY

Rev	Date	Rev by / Req by	Purpose
1.0	4/11/2006	Kevin Guerra	First draft.

TABLE OF CONTENTS

1.0	Introduction.....	4
1.1	Scope.....	4
1.2	Project Description.....	4
2.0	Applicable Documents.....	6
3.0	Component Layout.....	7
4.0	Error Handling Responsibilities per Module.....	8
4.1	BLL Module.....	8
4.2	SAL Module.....	8
4.3	SWIG Interface Module.....	8
4.4	Target Scripting Language Library Module.....	8
4.5	Limitations.....	8
4.6	Possible Future Enhancements.....	9
5.0	Design Requirements.....	10
5.1	Tools.....	10
5.2	Dependencies.....	10
6.0	Unit Test Strategy.....	11
7.0	Requirements Conformance.....	12
8.0	Acronyms.....	13
9.0	Open Issues.....	14
10.0	Appendix.....	15

1.0 Introduction

1.1 Scope

This document will detail the error handling strategy adopted by the automation team for the SAL project and other interacting modules.

1.2 Project Description

The goal of this project is to have a uniform way of handling errors across Spirent's Automation Layer and its modules. The challenge is to consolidate differing error handling strategies adopted by other teams and other tools that our modules use.

In our modules we must have code that deals with such differences in error handling by paying close attention to all function calls and handle the errors appropriately. To our users, however, these differences must be transparent and only one way of dealing with errors must be adopted.

We have decided to use C++ exceptions for dealing with most if not all the code produced by our team. Other teams also use this strategy. The C++ standard libraries also use exceptions and only very few sections of the code use "C" style error handling code.

The basic idea is to let all exceptions "pass through" the code until they are caught in only one location. Unless the code can recover from the error, in which case the exception should be caught there and error recovery code should also be provided, there. C style errors will be checked for and an exception will be raised. When these exceptions get to the exception handling code, they will all be converted to a "standard" C++ exception and it will then be delegated, through the SWIG Interface DLL, to the user's script. For more information on this, please review document "P2-Sal Interface Wrappers.doc"

2.0 Applicable Documents

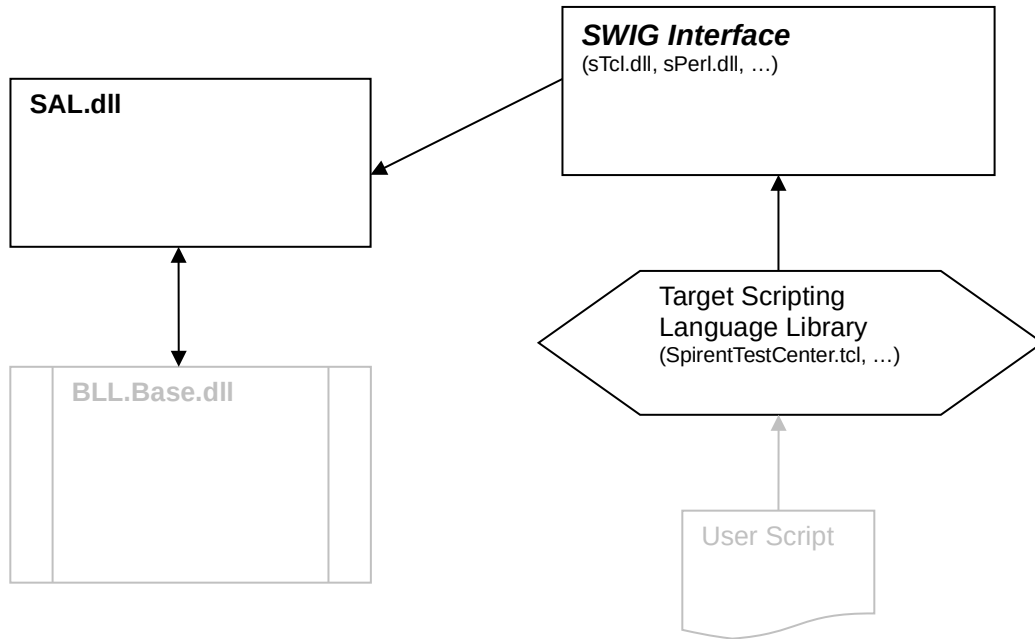
Table 2-1 lists the applicable documents referenced by this document. In case of conflict, this document takes precedence.

Table 2-1. Applicable Documents

Document Number	Title
	P2-Sal Interface Wrappers.doc

3.0 Component Layout

The following diagram contains the modules that the Automation team uses to provide an interface to users:



4.0 Error Handling Responsibilities per Module

4.1 BLL Module

This module can generate standard C++ exceptions, STL exceptions and StcExceptions. Some classes from this module have functions that return error codes. The messaging system that is used to interact with this module can also report errors, and these errors have to be decoded in the message decoding functions.

4.2 SAL Module

This is our main module in which we centralize our error handling code. This is the basic strategy:

1. Any errors generated here are communicated via SalException. Again, we should attempt to recover from an error when practical.
2. Any exceptions generated in an external class are let through until it reaches the error handler routine, which is in SalInterface.cpp.
3. When a function returns an error code, our code will raise an exception, but only if we cannot recover from the error condition. This is true for external classes or external functions.
4. When a C function is executed, the global errno should be checked, if in error, an exception will be raised, but only if we cannot recover from the error condition.
5. The message service class will decode any message response from the BLL and also decode event notifications from the BLL. These responses may contain error notifications, when this happens, and when applicable, we will raise a SalException.

The challenge here is to be able to catch any and all exceptions and error conditions in the error handling routines, which are part of “all” functions in the API. The API consists of all exportable functions in SalInterface.cpp.

SalInterface.cpp API functions have try-catch clauses that handle all exceptions. Any exception is then converted into a C++ standard exception. The reason for this is that SWIG can only handle standard C++ exceptions.

4.3 SWIG Interface Module

All this module does is catch exceptions and delegates them to the caller. In this case the caller is the target language specific library, e.g. SpirentTestCenter.tcl

This module, by the way, has the potential of raising its own exceptions in the wrapper code. This module will raise exceptions when incorrect or incomplete parameters are used. Of course, there are other reasons this module can also raise exceptions.

4.4 Target Scripting Language Library Module

As explained in document “P2-Sal Interface Wrappers.doc,” This is the place where we need to wrap and bullet proof any API function that we “do not” want it to crash when an exception occurs. For any other function, they are left “as is” and when an exception happens, they will stop the script and output the error to STDERR.

4.5 Limitations

Currently we handle errors for our code only. However, because our code uses code from other tools, such as Log4CPP for example, that code has the potential for failure. When that code fails, it can have any number of consequences. It can either, not affect the runtime, slightly affect it, or be

catastrophic, such as a program crash. Usually, modern code uses exceptions to communicate errors; in this case all we have to do is enclose the code in a try-catch clause. And we do so, since we wrap our code in the `SalInterface.cpp` file in that said clause.

The limitation in this case is the potential loss of context in which the error occurred. However, this problem appears to be minor, since that would usually be in code that is not ours. It could be time consuming to put those types of errors in context anyway. Suffice it to say that there was an error, and usually we will know where it came from, but may be not exactly where or why.

4.6 Possible Future Enhancements

It is advised by several design books that an error handling strategy not be an after thought. But it be a part of the system architecture. Consistency should certainly be a goal, but this may not be so easy to achieve.

There are several classes that return error codes, and of course the better alternative would be to use exceptions. Also C style functions do not help the matter, so anytime that a programmer would use a C function should first consider carefully if a similar function is available in C++.

There are also some issues with the exception handling mechanism. By default, a class member function is given the ability by the compiler to raise any kind of exception. This implies that there is a performance tradeoff. When a function is declared to not throw exceptions, the programmer must be extremely careful that they in fact do not. Otherwise, a violation of this rule will result in a program crash.

Another issue is that in the exception handling code, we must have a place where we are able to catch any type of exception. Otherwise, an uncaught exception will also cause a program to crash. For our project we seem to be protected from that eventuality, since in the API error handling code, we handle all types of exceptions eventually in a “catch(...)” clause. This provides the safety net that we need.

The drawback to this strategy is that it may lack completeness, in that, for some code, it may be helpful to be able to recover from an error right then and there. In that case, it is necessary to have a try-catch clause for the exceptions that we desire to handle in “that” code. This way, our application can be more resilient.

In the case of user errors, there probably won't be very much we can do but to report the error, in hopes that the user will know how to fix it. However, for other errors, such as internal errors, there may be cases where we can do a better job of recovering from the error.

5.0 Design Requirements

The requirement is to use standard components almost exclusively and to integrate other legacy methods into the strategy transparently.

In using standard components and modern style error handling mechanisms, we arguably produce better performing code, and without a doubt, better quality code. The code produced is not cluttered by error handling code in which we check for function error return values all over the place, but they are handled in usually a centralized location.

5.1 Tools

No tools other than the standard development tools and the tools used by the projects were necessary. That is, nothing was specifically required by the error handling mechanism, per se.

5.2 Dependencies

SAL depends on some classes from BLL.Base. Access to BLL.Base include and library directories is necessary. Specifically, the messaging service classes, exception classes, and some included files such as FrameworkConst.h.

6.0 Unit Test Strategy

We are to test every single public function as a matter of policy. This will ensure that code we expect to execute will be tested. In a system in which exception handling code is used, it is necessary to be able to cause exceptions to happen, so that the code is tested properly.

This is a significant issue, and the difficulty will be the thoroughness needed and the number of tests cases to be able to trigger every possible exception. Some are easy to trigger, but some are more elaborate and time consuming in terms of runtime. For example, how do you test a low memory condition? In that case you must use tools or other ways to deplete system memory and then allow the system to be tested.

For our test cases we have currently in other parts of the project, we have observed that a good number of test cases including negative tests cases which cause exceptions give us about 87% code coverage. The practical goal has been to place coverage at about 75%. So for all practical purposes, coding test cases for “most” exceptions seems to hit and surpass that mark. And yet, our aim should be to have 100% coverage.

7.0 Requirements Conformance

ERQ (Priority, Phase)	Description	Section Addressed	Comment
TBA			

8.0 Acronyms

BLL	Business Logic Layer
ERD	Engineering Requirements Document
ERQ	Engineering Requirement
SAL	Spirent Automation Layer
SWIG	Simple Wrapper Interface Generator
TBA	To be added

9.0 Open Issues

TBA, as needed.

Issue	Description	Status

10.0 Appendix